

DOKUMENTASI TEKNIS

Modul Odoo: database_siswa

PT Koding Yuk Academy

Versi Modul: 17.0.1.0.2 | Platform: Odoo 17

Tanggal Dokumen: 17 May 2026

1. Overview Modul

Modul database_siswa adalah modul kustom Odoo 17 yang dikembangkan oleh PT Koding Yuk Academy untuk mengelola seluruh data akademik siswa, mulai dari profil siswa, pendaftaran kursus, pelaksanaan ujian, penilaian sertifikat, hingga portofolio karya siswa. Modul ini juga menyediakan REST API berbasis JSON untuk keperluan Student Dashboard eksternal.

1.1 Informasi Modul

Atribut	Nilai
Nama Modul	Database Siswa
Technical Name	database_siswa
Versi	17.0.1.0.2
Kategori	Education
Author	PT Koding Yuk Academy
Website	https://kodingyuk.id
Lisensi	LGPL-3
Platform	Odoo 17
Dependensi	base, mail, contacts
Tipe Aplikasi	Application (muncul di menu utama)

1.2 Struktur Direktori

Berikut adalah struktur direktori lengkap modul database_siswa:

```
database_siswa/
├── __init__.py                # Entry point modul
├── __manifest__.py            # Konfigurasi & metadata modul
├── controllers/
│   ├── __init__.py
│   └── student_api.py         # REST API Controller (Student Dashboard)
├── lib/
│   ├── __init__.py
│   └── firebase_service.py    # Service layer Firebase Storage
├── models/
│   ├── __init__.py
│   ├── m_siswa.py             # Model Profil Siswa
│   ├── m_enrollment.py        # Model Pendaftaran Kursus
│   ├── m_class_type.py        # Master Jenis Kelas
│   ├── m_level_siswa.py       # Master Level Siswa
│   ├── m_exam_siswa.py        # Model Ujian & Detail Jawaban
│   ├── m_penilaian_sertifikat.py # Model Penilaian Sertifikat
│   ├── m_portfolio.py         # Model Portofolio Karya Siswa
│   └── m_invoice_automation.py # Cron Job Invoice Otomatis
├── security/
│   ├── security.xml           # Definisi grup akses
│   └── ir.model.access.csv     # Hak akses per model
├── views/
│   ├── m_siswa_views.xml
│   ├── m_enrollment_views.xml
│   ├── m_exam_siswa_views.xml
│   ├── m_penilaian_sertifikat_views.xml
│   ├── m_level_siswa_views.xml
│   ├── m_class_type_views.xml
│   ├── automation_cron.xml     # Konfigurasi Cron Job
│   └── student_menus.xml       # Definisi menu navigasi
├── wizard/
│   ├── __init__.py
│   ├── exam_start_wizard.py    # Wizard mulai ujian
│   └── exam_start_wizard_views.xml
└── static/
    └── description/
        └── index.html          # Deskripsi modul di App Store Odoo
```

2. Arsitektur & Entity Relationship Diagram (ERD)

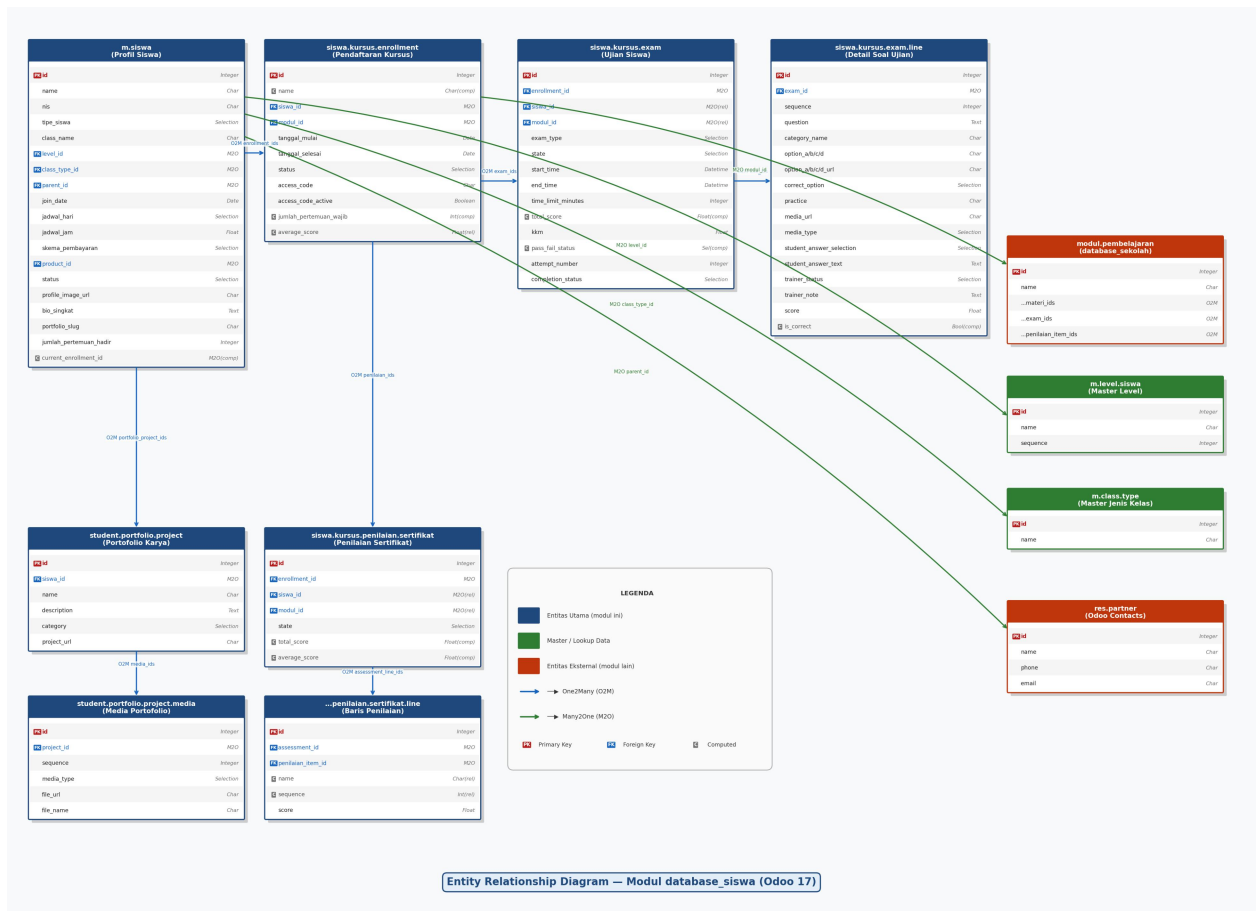
2.1 Daftar Model (Tabel Database)

Berikut adalah seluruh model (tabel) yang didefinisikan dalam modul database_siswa beserta nama teknis dan deskripsinya:

No	Nama Model (Python Class)	Technical Name (DB Table)	Deskripsi
1	StudentProfile	m.siswa (m_siswa)	Profil utama siswa, menyimpan data personal, level, jenis kelas, jadwal, dan relasi ke orang tua.
2	StudentCourseEnrollment	siswa.kursus.enrollment (siswa_kursus_enrollment)	Pendaftaran siswa ke kursus/modul tertentu. Menyimpan kode akses ujian dan status kelulusan.
3	StudentLevel	m.level.siswa (m_level_siswa)	Master data tingkat/level siswa (Builder, Creator, dll.).
4	StudentClassType	m.class.type (m_class_type)	Master data jenis kelas (Merakit, Scratch Jr, dll.).
5	SiswaKursusExam	siswa.kursus.exam (siswa_kursus_exam)	Record ujian siswa per enrollment. Menyimpan tipe ujian, waktu, skor, dan status.
6	SiswaKursusExamLine	siswa.kursus.exam.line (siswa_kursus_exam_line)	Detail baris soal ujian (snapshot soal + jawaban siswa + penilaian trainer).
7	SiswaKursusPenilaianSertifikat	siswa.kursus.penilaian.sertifikat (siswa_kursus_penilaian_sertifikat)	Penilaian sertifikat kelulusan kursus siswa.
8	SiswaKursusPenilaianSertifikatLine	siswa.kursus.penilaian.sertifikat.line (siswa_kursus_penilaian_sertifikat_line)	Baris detail penilaian sertifikat per item penilaian.
9	StudentPortfolioProject	student.portfolio.project (student_portfolio_project)	Karya/proyek portofolio siswa.
10	StudentPortfolioProjectMedia	student.portfolio.project.media (student_portfolio_project_media)	Media (foto/video) yang terkait dengan karya portofolio.
11	ExamStartWizard	siswa.kursus.exam.start.wizard (transient)	Wizard sementara untuk memilih tipe ujian yang akan dimulai.

2.2 Relasi Antar Model (ERD Tekstual)

Diagram berikut menggambarkan relasi antar entitas utama dalam modul database_siswa:



Gambar 1. Entity Relationship Diagram — Modul database_siswa

2.3 Detail Field Per Model

2.3.1 Model: m.siswa (Profil Siswa)

Field	Tipe	Keterangan	Constraint
id	Integer (auto)	Primary Key	PK, Auto
name	Char	Nama lengkap siswa	Required, Unique(name+parent_id)
nis	Char	Nomor Induk Siswa	-
tipe_siswa	Selection	privat / online / reguler / ekskul	Required, Default: privat
class_name	Char	Nama kelas (misal: 2 SD, TK B)	-
level_id	Many2one → m.level.siswa	Level/tingkat siswa	-
class_type_id	Many2one → m.class.type	Jenis kelas	-
join_date	Date	Tanggal pertama masuk	Default: today
jadwal_hari	Selection	Hari jadwal kelas	-
jadwal_jam	Float	Jam jadwal kelas	-
skema_pembayaran	Selection	monthly / semester	Required, Default: monthly
product_id	Many2one → product.product	Produk untuk penagihan invoice	-
parent_id	Many2one → res.partner	Penanggung jawab / orang tua	-
parent_contact	Char (related)	Nomor HP dari parent_id.phone	Readonly

status	Selection	active / leave / graduated / inactive	Default: active
notes	Text	Keterangan tambahan	-
image_1920	Image	Foto profil (binary, max 1920px)	-
profile_image_url	Char	URL foto profil di Firebase Storage	-
bio_singkat	Text	Bio untuk halaman portofolio	-
portfolio_slug	Char	URL slug portofolio (misal: aril-saputra)	-
jumlah_pertemuan_hadir	Integer	Total pertemuan yang dihadiri	Readonly, Default: 0
jumlah_pertemuan_ditagih	Integer	Total pertemuan yang sudah ditagih	Readonly, Default: 0
enrollment_ids	One2many → siswa.kursus.enrollment	Riwayat kursus siswa	-
portfolio_project_ids	One2many → student.portfolio.project	Daftar karya portofolio	-
current_enrollment_id	Many2one (computed)	Kursus aktif saat ini	Computed, Store

2.3.2 Model: siswa.kursus.enrollment (Pendaftaran Kursus)

Field	Tippe	Keterangan	Constraint
id	Integer (auto)	Primary Key	PK, Auto
name	Char (computed)	Format: "Nama Siswa - Nama Modul"	Computed, Store
siswa_id	Many2one → m.siswa	Siswa yang mendaftar	Required, ondelete=cascade
modul_id	Many2one → modul.pembelajaran	Kursus/modul yang diikuti	Required
tanggal_mulai	Date	Tanggal mulai kursus	Required, Default: today
tanggal_selesai	Date	Tanggal selesai kursus	-
status	Selection	aktif / lulus / berhenti	Required, Default: aktif
access_code	Char	Kode akses ujian (format: EXM-XXXXXX)	Readonly, Copy=False
access_code_active	Boolean	Status aktif kode akses	Default: False
jumlah_pertemuan_wajib	Integer (computed)	Jumlah materi di modul	Computed, Store
jumlah_pertemuan_diikuti	Integer (computed)	Pertemuan yang diikuti siswa	Computed, Store
average_score	Float (related)	Rata-rata nilai dari penilaian sertifikat	Related, Store
exam_ids	One2many → siswa.kursus.exam	Daftar ujian siswa	-
penilaian_ids	One2many → siswa.kursus.penilaian.sertifikat	Penilaian sertifikat	-

SQL Constraint: unique(siswa_id, modul_id) — Satu siswa hanya bisa mendaftar satu kali per modul.

2.3.3 Model: siswa.kursus.exam (Ujian Siswa)

Field	Tippe	Keterangan	Constraint
id	Integer (auto)	Primary Key	PK, Auto
enrollment_id	Many2one → siswa.kursus.enrollment	Pendaftaran kursus terkait	Required, ondelete=cascade
siswa_id	Many2one (related)	Siswa (dari enrollment)	Related, Store, Readonly
modul_id	Many2one (related)	Modul (dari enrollment)	Related, Store, Readonly
tanggal_ujian	Date	Tanggal pelaksanaan ujian	Default: today
exam_type	Selection	pilihan_ganda / essai / praktik	Required
state	Selection	draft / in_progress /	Default: draft

		submitted / done	
completion_status	Selection	done / timeout	Copy=False
start_time	Datetime	Waktu mulai ujian	Copy=False
end_time	Datetime	Waktu selesai ujian	Copy=False
time_limit_minutes	Integer	Batas waktu ujian (menit)	Default: 0
total_score	Float (computed)	Skor akhir ujian	Computed, Store
kkm	Float	Nilai Kriteria Ketuntasan Minimal	Default: 75.0
pass_fail_status	Selection (computed)	pass / fail	Computed, Store
attempt_number	Integer	Percobaan ke-berapa	Default: 1
display_name	Char (computed)	Nama tampilan ujian	Computed, Store
line_ids	One2many → siswa.kursus.exam.line	Detail soal & jawaban	-

2.3.4 Model: siswa.kursus.exam.line (Detail Soal Ujian)

Field	Tipe	Keterangan	Constraint
id	Integer (auto)	Primary Key	PK, Auto
exam_id	Many2one → siswa.kursus.exam	Ujian induk	onDelete=cascade
sequence	Integer	Nomor urut soal	-
question	Text	Teks pertanyaan (snapshot dari modul)	-
category_name	Char	Nama kategori soal (snapshot)	-
option_a/b/c/d	Char	Teks pilihan jawaban A-D	-
option_a/b/c/d_url	Char	URL gambar pilihan jawaban A-D	-
correct_option	Selection (A/B/C/D)	Jawaban benar (snapshot, tersembunyi)	-
practice	Char	Instruksi praktik (snapshot)	-
description	Text	Deskripsi soal (snapshot)	-
media_url	Char	URL media soal (gambar/video)	-
media_type	Selection	image / video / other	-
student_answer_selection	Selection (A/B/C/D)	Jawaban siswa untuk PG	-
student_answer_text	Text	Jawaban siswa untuk Essai	-
trainer_status	Selection	berhasil / gagal (untuk Praktik)	-
trainer_note	Text	Catatan dari trainer/mentor	-
score	Float	Skor untuk baris ini	-
is_correct	Boolean (computed)	Apakah jawaban PG benar?	Computed, Store

2.3.5 Model: siswa.kursus.penilaian.sertifikat (Penilaian Sertifikat)

Field	Tipe	Keterangan	Constraint
id	Integer (auto)	Primary Key	PK, Auto
enrollment_id	Many2one → siswa.kursus.enrollment	Pendaftaran kursus terkait	Required, onDelete=cascade
siswa_id	Many2one (related)	Siswa (dari enrollment)	Related, Store
modul_id	Many2one (related)	Modul (dari enrollment)	Related, Store
state	Selection	draft / done	Default: draft
total_score	Float (computed)	Jumlah total skor semua item	Computed, Store
average_score	Float (computed)	Rata-rata skor	Computed, Store
assessment_line_ids	One2many → ...sertifikat.line	Detail item penilaian	-

2.3.6 Model: student.portfolio.project (Portofolio Karya)

Field	Tipe	Keterangan	Constraint
id	Integer (auto)	Primary Key	PK, Auto
siswa_id	Many2one → m.siswa	Pemilik karya	Required, ondelete=cascade
name	Char	Judul proyek/karya	Required
description	Text	Deskripsi proyek	-
category	Selection	scratch/pictoblox/arduino/python/roblox/minecraft/web/other	Required
project_url	Char	URL live project atau download link	-
media_ids	One2many → ...project.media	Media foto/video karya	-

3. Fungsi Tiap File

3.1 __manifest__.py

File konfigurasi utama modul Odoo. Mendefinisikan metadata modul (nama, versi, author, kategori, lisensi), daftar dependensi modul lain, dan urutan loading file data (security, wizard, views, menus). File ini wajib ada di setiap modul Odoo.

3.2 __init__.py (root)

Entry point modul. Mengimpor sub-paket models, controllers, dan wizard agar Odoo dapat menemukan dan mendaftarkan semua class model dan controller yang ada di dalamnya.

3.3 controllers/student_api.py

File ini berisi class StudentExamAPIController yang merupakan REST API controller untuk Student Dashboard eksternal. Semua endpoint menggunakan type="json" (JSON-RPC style), auth="public" (tidak memerlukan login Odoo), dan CORS terbuka. Autentikasi dilakukan melalui access_code yang dikirim di setiap request.

Metode-metode dalam file ini:

Metode	Fungsi	Endpoint
_validate_access_code()	Helper private: memvalidasi access_code dan mengembalikan record enrollment yang aktif.	(internal)
student_login()	Validasi kode akses siswa, mengembalikan data enrollment, profil siswa, dan daftar ujian beserta sisa waktu.	POST /api/v1/student/login
student_dashboard_data()	Mengambil data lengkap dashboard: profil, ringkasan absensi, riwayat absensi, performa/penilaian, dan daftar ujian.	POST /api/v1/student/dashboard_data
exam_detail()	Mengambil detail soal-soal ujian tertentu beserta jawaban siswa yang sudah diisi.	POST /api/v1/student/exam/detail
exam_start()	Memulai ujian: mengubah state dari draft ke in_progress dan mencatat start_time.	POST /api/v1/student/exam/start
exam_submit()	Menyimpan jawaban siswa dan menyelesaikan ujian (action_done).	POST /api/v1/student/exam/submit
exam_done()	Menandai ujian selesai dengan status timeout (digunakan saat waktu habis dari sisi client).	POST /api/v1/student/exam/done
time_config()	Mengambil konfigurasi durasi waktu ujian per tipe ujian dari model exam.time.config.	POST /api/v1/student/time-config

3.4 lib/firebase_service.py

Service layer untuk integrasi dengan Firebase Storage (Google Cloud). Menggunakan Firebase Admin SDK. Konfigurasi (service account key dan bucket name) disimpan di ir.config_parameter Odoo, bukan di file kode.

Fungsi	Deskripsi
get_firebase_app(env)	Menginisialisasi dan mengembalikan instance Firebase Admin SDK. Menggunakan singleton pattern

	(<code>_firebase_app</code>) agar tidak re-inisialisasi berulang kali. Membaca kredensial dari <code>ir.config_parameter</code> .
<code>upload_file_to_firebase(env, file_content, file_name, destination_path, content_type)</code>	Mengupload file (bytes) ke Firebase Storage pada path yang ditentukan. Mengatur file menjadi public dan mengembalikan URL publik.
<code>delete_file_from_firebase(env, file_path)</code>	Menghapus file dari Firebase Storage berdasarkan path. Mengembalikan True jika berhasil, False jika file tidak ditemukan.

Konfigurasi Firebase di `ir.config_parameter`:

- `firebase.service.account.key` → JSON string service account key
- `firebase.storage.bucket.name` → Nama bucket Firebase Storage (misal: `my-app.appspot.com`)

3.5 models/m_siswa.py

Mendefinisikan model `StudentProfile` (`m.siswa`) sebagai entitas utama siswa. Mewarisi `mail.thread` dan `mail.activity.mixin` untuk fitur chatter dan log histori. Menangani upload foto profil ke Firebase Storage secara otomatis saat `create/write`.

Metode	Deskripsi
<code>_compute_current_enrollment()</code>	Computed field: mencari enrollment dengan status "aktif" dan menyimpannya sebagai kursus aktif saat ini.
<code>create(vals)</code>	Override create: setelah record dibuat, jika ada <code>image_1920</code> , otomatis upload ke Firebase.
<code>write(vals)</code>	Override write: setelah record diupdate, jika ada perubahan <code>image_1920</code> , otomatis upload ke Firebase.
<code>_upload_image_to_bucket()</code>	Helper: decode base64 image, upload ke Firebase Storage di path <code>students/profiles/{id}/</code> , simpan URL, lalu hapus binary dari DB untuk efisiensi.

3.6 models/m_enrollment.py

Mendefinisikan model `StudentCourseEnrollment` (`siswa.kursus.enrollment`) yang merepresentasikan pendaftaran seorang siswa ke satu kursus/modul. Model ini menjadi hub utama yang menghubungkan siswa, modul, ujian, dan penilaian sertifikat.

Metode	Deskripsi
<code>_compute_name()</code>	Computed: menghasilkan nama format "Nama Siswa - Nama Modul".
<code>_compute_jumlah_pertemuan_wajib()</code>	Computed: menghitung jumlah materi di modul sebagai target pertemuan wajib.
<code>_compute_jumlah_pertemuan_diikuti()</code>	Computed: placeholder, diisi oleh modul <code>absensi_siswa</code> via extension.
<code>action_generate_access_code()</code>	Generate kode akses ujian format EXM-XXXXXX menggunakan UUID, aktifkan kode, tampilkan notifikasi.
<code>action_deactivate_access_code()</code>	Menonaktifkan kode akses ujian (<code>access_code_active = False</code>).
<code>action_start_exam(selected_types)</code>	Membuat record ujian baru dari template di modul, mengacak urutan soal, dan generate kode akses jika belum ada.
<code>action_view_student_exams()</code>	Membuka list view ujian yang difilter berdasarkan enrollment ini.
<code>action_create_or_view_certificate_assessment()</code>	Membuat atau membuka penilaian sertifikat untuk enrollment ini. Jika belum ada, buat baru dan pre-fill dari template modul.
<code>_compute_has_certificate_assessment()</code>	Computed: cek apakah sudah ada penilaian sertifikat untuk enrollment ini.

3.7 models/m_exam_siswa.py

Mendefinisikan dua model: SiswaKursusExam (ujian) dan SiswaKursusExamLine (detail soal). Model ujian mengelola lifecycle ujian dari draft hingga done, menghitung skor otomatis, dan menentukan status lulus/tidak lulus berdasarkan KKM.

Metode	Deskripsi
_compute_display_name()	Computed: format "Ujian {Tipe} - {Nama Enrollment} (Percobaan #{N})".
_compute_total_score()	Computed: untuk PG = (benar/total)*100; untuk Essai/Praktik = rata-rata skor baris. Juga menentukan pass_fail_status vs KKM.
action_start()	Mengubah state ke in_progress, mencatat start_time, set time_limit_minutes default 30 menit jika belum diset.
action_done(status)	Mengakhiri ujian: catat end_time, set completion_status. PG langsung ke "done", Essai/Praktik ke "submitted" (menunggu penilaian trainer).
action_trainer_done()	Trainer menandai ujian Essai/Praktik sebagai selesai dinilai (state = done).
action_reset_to_draft()	Reset ujian ke draft untuk percobaan ulang: hapus start/end time, reset semua jawaban siswa.
SiswaKursusExamLine._compute_is_correct()	Computed: membandingkan student_answer_selection dengan correct_option untuk tipe PG.
SiswaKursusExamLine._compute_media_preview()	Computed: menghasilkan HTML preview untuk media (img tag untuk gambar, video tag untuk video).
SiswaKursusExamLine.action_open_media()	Membuka URL media di tab baru browser.

3.8 models/m_penilaian_sertifikat.py

Mendefinisikan model penilaian sertifikat kelulusan kursus. Saat trainer menyelesaikan penilaian (action_set_done), status enrollment siswa otomatis berubah menjadi "lulus".

3.9 models/m_portfolio.py

Mendefinisikan model portofolio karya siswa. Setiap karya dapat memiliki beberapa media (foto/video). File media diupload otomatis ke Firebase Storage saat create/write, mirip dengan mekanisme upload foto profil di m_siswa.py.

3.10 models/m_invoice_automation.py

Meng-extend model m.siswa dengan menambahkan metode _cron_auto_create_invoice() yang dijalankan oleh Cron Job Odoo. Metode ini memeriksa semua siswa aktif dan membuat invoice otomatis ke orang tua/penanggung jawab berdasarkan skema pembayaran (monthly = per 4 pertemuan, semester = per 12 pertemuan).

3.11 wizard/exam_start_wizard.py

Wizard transient untuk memilih tipe ujian yang akan dimulai (Pilihan Ganda, Essai, Praktik). Secara otomatis pre-select tipe ujian berdasarkan template yang ada di modul, dan tidak menampilkan tipe yang sudah lulus. Saat dikonfirmasi, membuat record ujian baru dengan soal yang diacak dari template modul.

3.12 models/m_level_siswa.py & m_class_type.py

m_level_siswa.py: Master data tingkat/level siswa (Builder, Creator, dll.) dengan field name dan sequence untuk pengurutan. Memiliki constraint unique pada name.

m_class_type.py: Master data jenis kelas (Merakit, Scratch Jr, dll.) dengan field name. Memiliki constraint unique pada name.

4. API Reference — Student Dashboard

Semua endpoint API menggunakan protokol JSON-RPC Odoo (type="json"). Request dikirim sebagai HTTP POST dengan Content-Type: application/json. Body request harus mengikuti format JSON-RPC 2.0 dengan params berisi parameter endpoint.

4.1 Format Request & Response Umum

Format Request (JSON-RPC)

```
POST /api/v1/student/<endpoint>
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "method": "call",
  "params": {
    "access_code": "EXM-XXXXXX",
    ... parameter lainnya ...
  }
}
```

Format Response Sukses

```
{
  "jsonrpc": "2.0",
  "id": null,
  "result": {
    "success": true,
    ... data response ...
  }
}
```

Format Response Error

```
{
  "jsonrpc": "2.0",
  "id": null,
  "result": {
    "success": false,
    "error": "Pesan error dalam Bahasa Indonesia"
  }
}
```

4.2 Mekanisme Autentikasi

Semua endpoint menggunakan auth="public" (tidak memerlukan session Odoo). Autentikasi dilakukan melalui access_code yang dikirim di setiap request. Kode akses divalidasi terhadap field access_code dan access_code_active=True pada model siswa.kursus.enrollment.

Format Kode Akses: EXM-XXXXXX (prefix EXM- diikuti 6 karakter hex uppercase)
Contoh: EXM-A3F9B2
Kode akses di-generate via action_generate_access_code() di model enrollment.

4.3 POST /api/v1/student/login

Memvalidasi kode akses dan mengembalikan data awal untuk Student Dashboard.

Request Parameters

Parameter	Tipe	Required	Keterangan
access_code	string	Ya	Kode akses ujian siswa (case-insensitive, akan di-

			uppercase)
--	--	--	------------

Response Fields

```
{
  "success": true,
  "enrollment": {
    "id": 1,
    "name": "Budi Santoso - Python Dasar",
    "modul_name": "Python Dasar",
    "modul_id": 5,
    "status": "aktif"
  },
  "student": {
    "id": 3,
    "name": "Budi Santoso"
  },
  "exams": [
    {
      "id": 10,
      "display_name": "Ujian Pilihan Ganda - Budi Santoso - Python Dasar (Percobaan
#1)",
      "exam_type": "pilihan_ganda",
      "tanggal_ujian": "2024-01-15",
      "total_score": 85.0,
      "state": "done",
      "start_time": "2024-01-15 08:00:00",
      "time_limit_minutes": 30,
      "remaining_seconds": 0
    }
  ]
}
```

4.4 POST /api/v1/student/dashboard_data

Mengambil data lengkap untuk halaman dashboard siswa.

Request Parameters

Parameter	Tipe	Required	Keterangan
access_code	string	Ya	Kode akses ujian siswa

Response Fields

```
{
  "success": true,
  "profile": {
    "id": 3, "name": "Budi Santoso", "nis": "2024001",
    "class_name": "5 SD", "level_name": "Builder",
    "tipe_siswa": "reguler", "join_date": "2024-01-01",
    "status": "active", "bio": "Suka coding Python!"
  },
  "attendance_summary": {
    "total_hadir": 8, "total_izin": 1, "total_absen": 0,
    "total_pertemuan": 12, "pertemuan_ke_berapa": 9
  },
  "attendance_history": [
    {
      "id": 1, "pertemuan_ke": 1,
      "display_name": "Pertemuan 1 - Python Dasar",
      "status": "hadir",
      "tanggal_waktu": "2024-01-08 09:00:00",
      "notes": ""
    }
  ],
  "performance": {
```

```

        "total_score": 270.0, "average_score": 90.0,
        "state": "done",
        "lines": [
            {"name": "Pemahaman Konsep", "score": 90.0},
            {"name": "Praktik Coding", "score": 85.0}
        ]
    },
    "exams": [ ... (sama dengan response login) ... ],
    "enrollment": { "id": 1, "name": "...", "modul_name": "...", "status": "aktif" }
}

```

4.5 POST /api/v1/student/exam/detail

Mengambil detail soal-soal ujian tertentu.

Request Parameters

Parameter	Tipe	Required	Keterangan
access_code	string	Ya	Kode akses ujian siswa
exam_id	integer	Ya	ID record ujian (siswa.kursus.exam)

Response Fields

```

{
    "success": true,
    "exam": {
        "id": 10, "display_name": "Ujian Pilihan Ganda - ...",
        "exam_type": "pilihan_ganda", "state": "in_progress",
        "total_score": 0.0, "start_time": "2024-01-15 08:00:00",
        "time_limit_minutes": 30, "remaining_seconds": 1500
    },
    "lines": [
        {
            "id": 101, "sequence": 1,
            "question": "Apa output dari print('Hello')?",
            "category_name": "Dasar Python",
            "option_a": "Hello", "option_b": "hello",
            "option_c": "HELLO", "option_d": "Error",
            "option_a_url": "", "option_b_url": "", "option_c_url": "", "option_d_url": "",
            "practice": "", "description": "", "media_url": "", "media_type": "",
            "student_answer_selection": "",
            "student_answer_text": "",
            "trainer_status": "", "trainer_note": "",
            "score": 0.0, "is_correct": false
        }
    ]
}

```

4.6 POST /api/v1/student/exam/start

Memulai ujian: mengubah state ke in_progress dan mencatat waktu mulai.

Request Parameters

Parameter	Tipe	Required	Keterangan
access_code	string	Ya	Kode akses ujian siswa
exam_id	integer	Ya	ID record ujian

Response Fields

```

{
    "success": true,
    "start_time": "2024-01-15 08:00:00",
    "time_limit_minutes": 30,

```

```
    "remaining_seconds": 1800
}
```

Catatan: Jika ujian sudah berstatus "done", endpoint ini mengembalikan error. Jika sudah "in_progress", endpoint mengembalikan sisa waktu yang tersisa.

4.7 POST /api/v1/student/exam/submit

Menyimpan jawaban siswa dan menyelesaikan ujian.

Request Parameters

Parameter	Tipe	Required	Keterangan
access_code	string	Ya	Kode akses ujian siswa
exam_id	integer	Ya	ID record ujian
answers	array	Ya	Array objek jawaban siswa

Format Array answers

```
// Untuk tipe pilihan_ganda:
"answers": [
  {"line_id": 101, "answer": "A"},
  {"line_id": 102, "answer": "C"},
  {"line_id": 103, "answer": "B"}
]

// Untuk tipe essai:
"answers": [
  {"line_id": 201, "answer": "Jawaban panjang siswa di sini..."},
  {"line_id": 202, "answer": "Jawaban soal kedua..."}
]
```

Response Fields

```
{
  "success": true,
  "total_score": 85.0
}
```

Logika penyelesaian:

- Pilihan Ganda → state langsung menjadi "done", skor dihitung otomatis.
- Essai / Praktik → state menjadi "submitted" (menunggu penilaian trainer).

4.8 POST /api/v1/student/exam/done

Menandai ujian selesai dengan status timeout. Dipanggil oleh client saat timer habis.

Request Parameters

Parameter	Tipe	Required	Keterangan
access_code	string	Ya	Kode akses ujian siswa
exam_id	integer	Ya	ID record ujian

Response Fields

```
{"success": true}
```

4.9 POST /api/v1/student/time-config

Mengambil konfigurasi durasi waktu ujian per tipe dari model exam.time.config.

Request Parameters

Tidak ada parameter yang diperlukan (selain format JSON-RPC standar).

Response Fields

```
{
  "success": true,
  "configs": [
    {"exam_type": "pilihan_ganda", "duration_minutes": 30},
    {"exam_type": "esai", "duration_minutes": 45},
    {"exam_type": "praktik", "duration_minutes": 60}
  ]
}
```

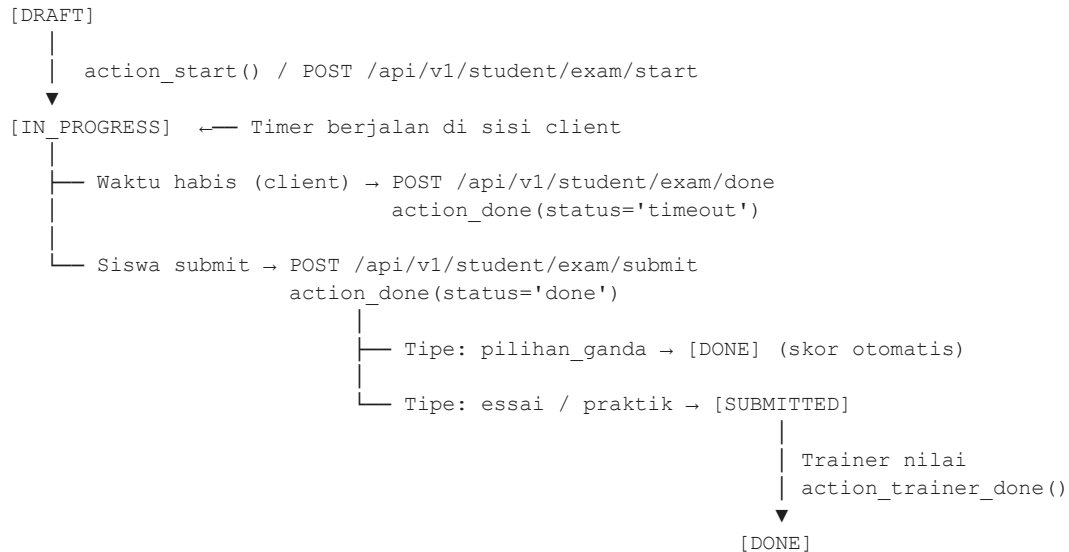
4.10 Ringkasan Endpoint API

No	Endpoint	Method	Auth	Fungsi
1	/api/v1/student/login	POST	access_code	Login & ambil data awal
2	/api/v1/student/dashboard_data	POST	access_code	Data lengkap dashboard
3	/api/v1/student/exam/detail	POST	access_code	Detail soal ujian
4	/api/v1/student/exam/start	POST	access_code	Mulai ujian
5	/api/v1/student/exam/submit	POST	access_code	Submit jawaban & selesaikan ujian
6	/api/v1/student/exam/done	POST	access_code	Tandai ujian selesai (timeout)
7	/api/v1/student/time-config	POST	public	Konfigurasi durasi ujian

5. Business Logic & Alur Proses

5.1 Alur Lifecycle Ujian Siswa

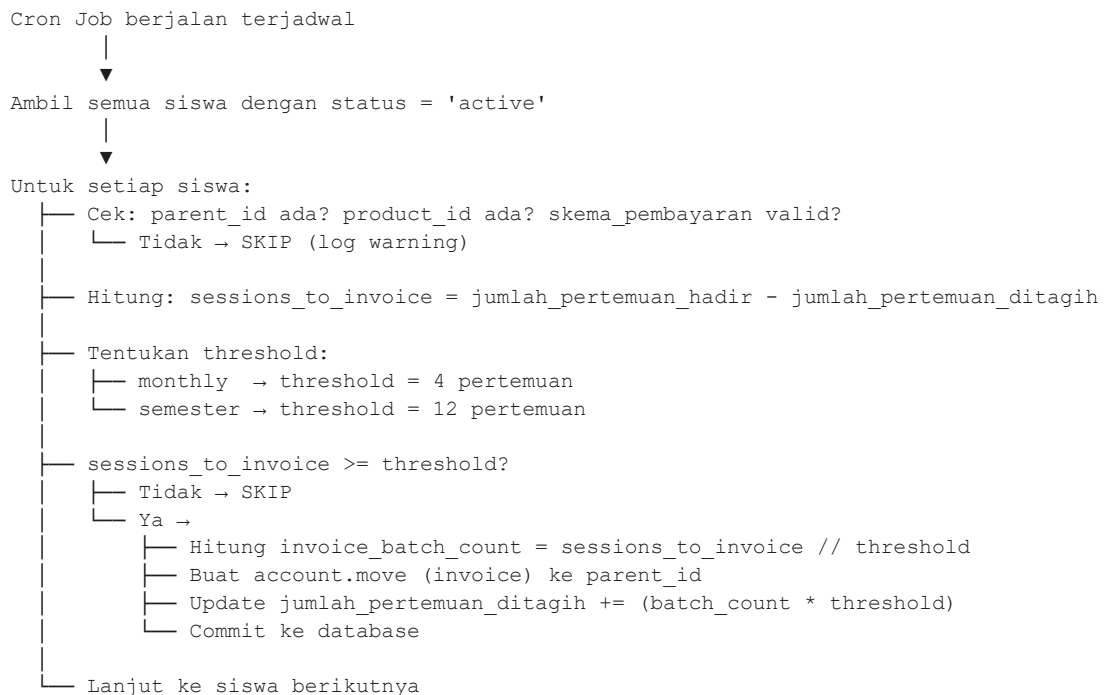
ALUR LIFECYCLE UJIAN (siswa.kursus.exam.state):



Untuk remedial: `action_reset_to_draft()` → kembali ke [DRAFT]

5.2 Alur Invoice Otomatis (Cron Job)

ALUR CRON JOB INVOICE OTOMATIS (_cron_auto_create_invoice):



5.3 Alur Upload Media ke Firebase Storage

ALUR UPLOAD FOTO PROFIL (m.siswa):



ALUR UPLOAD MEDIA PORTOFOLIO (student.portfolio.project.media):

Sama seperti di atas, path:

`'students/projects/{siswa_id}/project_{project_id}/{filename}'`

5.4 Alur Generate & Penggunaan Kode Akses

GENERATE KODE AKSES:

Admin/Trainer klik tombol "Generate Kode Akses" di form enrollment

`action_generate_access_code()`

`code = 'EXM-' + uuid4().hex[:6].upper()`

`enrollment.write({'access_code': code, 'access_code_active': True})`

Tampilkan notifikasi dengan kode akses

PENGUNAAN KODE AKSES (Student Dashboard):

Siswa input kode akses di halaman login dashboard

POST `/api/v1/student/login` dengan `{"access_code": "EXM-XXXXXX"}`

`_validate_access_code()` → search enrollment dengan:

- `access_code` = kode yang dikirim

- `access_code_active` = True

- └ Tidak ditemukan → `{"success": false, "error": "Kode akses tidak valid..."}`

- └ Ditemukan → return enrollment record → lanjut proses

6. Security & Hak Akses

6.1 Hak Akses Model (ir.model.access.csv)

Semua model diberikan hak akses penuh (read, write, create, delete) kepada group base.group_user (semua user Odoo yang login).

Model	Group	Read	Write	Create	Delete
m.siswa	base.group_user	✓	✓	✓	✓
siswa.kursus.enrollment	base.group_user	✓	✓	✓	✓
m.level.siswa	base.group_user	✓	✓	✓	✓
m.class.type	base.group_user	✓	✓	✓	✓
siswa.kursus.penilaian.sertifikat	base.group_user	✓	✓	✓	✓
siswa.kursus.penilaian.sertifikat.line	base.group_user	✓	✓	✓	✓
siswa.kursus.exam	base.group_user	✓	✓	✓	✓
siswa.kursus.exam.line	base.group_user	✓	✓	✓	✓
siswa.kursus.exam.start.wizard	base.group_user	✓	✓	✓	✓
student.portfolio.project	base.group_user	✓	✓	✓	✓
student.portfolio.project.media	base.group_user	✓	✓	✓	✓

6.2 Keamanan API Endpoint

Aspek	Implementasi
Autentikasi	Berbasis access_code per enrollment. Kode divalidasi di setiap request.
Otorisasi	Siswa hanya bisa mengakses data enrollment miliknya sendiri (filter by enrollment_id).
CSRF	Dinonaktifkan (csrf=False) karena API menggunakan JSON-RPC, bukan form HTML.
CORS	Terbuka (cors="*") untuk mendukung akses dari domain frontend yang berbeda.
Sudo	Semua query database menggunakan .sudo() untuk bypass record rules Odoo.
Input Validation	access_code di-strip dan di-uppercase sebelum divalidasi.
Error Handling	Semua endpoint dibungkus try-except, error di-log dan dikembalikan sebagai JSON.

7. Konfigurasi & Instalasi

7.1 Dependensi Python

Library	Versi	Kegunaan
firebase-admin	>=5.0	Integrasi Firebase Storage untuk upload media
Odoo 17 (base, mail, contacts)	Bawaan	Framework utama

```
# Install Firebase Admin SDK di server Odoo:
pip install firebase-admin
```

7.2 Konfigurasi Firebase (ir.config_parameter)

Masuk ke Odoo → Settings → Technical → Parameters → System Parameters, lalu tambahkan dua parameter berikut:

Key	Value	Keterangan
firebase.service.account.key	{ "type": "service_account", ... }	JSON string dari file service account Firebase
firebase.storage.bucket.name	my-project.appspot.com	Nama bucket Firebase Storage

7.3 Instalasi Modul

```
# 1. Salin folder database_siswa ke direktori addons Odoo
cp -r database_siswa /path/to/odoo/addons/

# 2. Restart Odoo server
sudo systemctl restart odoo

# 3. Aktifkan Developer Mode di Odoo
# Settings → Activate Developer Mode

# 4. Update Apps List
# Apps → Update Apps List

# 5. Install modul
# Apps → Cari "Database Siswa" → Install
```

8. Cron Job

8.1 Invoice Otomatis

Atribut	Nilai
Nama	Auto Create Student Invoice
Model	m.siswa
Metode	_cron_auto_create_invoice()
Interval	Dikonfigurasi di automation_cron.xml
Trigger	Otomatis terjadwal
Fungsi	Membuat invoice ke orang tua siswa berdasarkan jumlah pertemuan yang dihadiri

Logika Invoice:

- Skema monthly: invoice dibuat setiap 4 pertemuan dihadiri
- Skema semester: invoice dibuat setiap 12 pertemuan dihadiri
- Invoice dibuat ke res.partner (parent_id) menggunakan product_id yang dikonfigurasi di profil siswa
- Setelah invoice dibuat, jumlah_pertemuan_ditagih diupdate secara otomatis

9. Catatan Teknis & Integrasi Modul Lain

Modul Terkait	Relasi	Keterangan
database_sekolah	modul.pembelajaran (M2O dari enrollment.modul_id)	Menyediakan data kursus/modul, template ujian, dan item penilaian sertifikat.
absensi_siswa	absensi.siswa.absensi (diakses via API dashboard)	Menyediakan data kehadiran siswa yang ditampilkan di dashboard.
Odoo Accounting	account.move (dibuat oleh cron invoice)	Invoice otomatis dibuat ke modul akuntansi Odoo.
Odoo Contacts	res.partner (parent_id di m.siswa)	Data orang tua/penanggung jawab diambil dari modul Contacts.
Firebase Storage	External service via firebase_service.py	Penyimpanan foto profil dan media portofolio siswa.

*Dokumen ini dibuat secara otomatis dari source code modul database_siswa. Dihasilkan pada: 17 May 2026, 10:07.
Untuk pertanyaan teknis, hubungi tim developer PT Koding Yuk Academy.*